

UNDERACTUATED ROBOTICS

Algorithms for Walking, Running, Swimming, Flying, and Manipulation

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-11-11.

[How to cite these notes, use annotations, and give feedback.](#)

Note: These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

CHAPTER 16

Algorithms for Limit Cycles

 Launch in Deepnote

The discussion of walking and running robots in Chapter 4 motivated the notion of limit cycle stability. Linear systems are not capable of producing stable limit cycle behavior, so this rich topic is unique to nonlinear systems design and analysis. Furthermore, the tools that are required to design, stabilize, and verify limit cycles will have applicability beyond simple periodic motions.

The first natural question we must ask is, given a system $\dot{\mathbf{x}} = f(\mathbf{x})$, or a control system $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$, how do we go about finding periodic solutions which may be passively stable, open-loop stable, or stabilizable via closed-loop feedback? It turns out that the trajectory optimization tools that we developed already are very well suited to this task.

16.1 TRAJECTORY OPTIMIZATION

I introduced the trajectory optimization tools as means for optimizing a control trajectory starting from a particular known initial condition. But the fundamental idea of optimizing over individual trajectories of the system is useful more broadly. Even for a passive system, we can formulate the search for a periodic solution as an optimization over trajectories that satisfy the dynamic constraints and *periodicity constraints*, $\mathbf{x}[0] = \mathbf{x}[N]$:

$$\begin{aligned} \text{find}_{\mathbf{x}[\cdot]} \quad & \text{subject to} \quad \mathbf{x}[n+1] = f(\mathbf{x}[n]), \quad \forall n \in [0, N-1] \\ & \mathbf{x}[0] = \mathbf{x}[N]. \end{aligned}$$

Certainly we can add control inputs back into the formulation, too, but let's start with this simple case. Take a moment to think about the feasible solutions to this problem formulation. Certainly a fixed point $\mathbf{x}[n] = \mathbf{x}^*$ will satisfy the constraints; if we don't want these solutions to come out of the solver we might need to exclude them with constraints or add an objective that guides the solver towards the desired solutions. The other possible solutions are trajectories that are periodic in exactly N steps. That's pretty restrictive.

We can do better if we use the continuous-time formulations. For instance, in our introduction of [direct collocation](#), we wrote

$$\begin{aligned} \min_{\mathbf{x}[\cdot], \mathbf{u}[\cdot]} \quad & \ell_f(\mathbf{x}[N]) + \sum_{n=0}^{N-1} h_n \ell(\mathbf{x}[n], \mathbf{u}[n]) \\ \text{subject to} \quad & \dot{\mathbf{x}}(t_{c,n}) = f(\mathbf{x}(t_{c,n}), \mathbf{u}(t_{c,n})), \quad \forall n \in [0, N-1] \\ & \mathbf{x}[0] = \mathbf{x}_0 \\ & + \text{additional constraints.} \end{aligned}$$

But we can also add h_n as decision variables in the optimization (reminder: I recommend setting a lower-bound $h_n \geq h_{min} > 0$). This allows our N -step trajectory optimization to scale and shrink time in order to satisfy the periodicity constraint. The result is simple and powerful.

Example 16.1 ([Finding the limit cycle of the Van der Pol oscillator](#))

Recall the dynamics of the Van der Pol oscillator given by

$$\ddot{q} + \mu(q^2 - 1)\dot{q} + q = 0, \quad \mu > 0,$$

which exhibited a stable limit cycle.

Formulate the direct collocation optimization:

$$\begin{aligned} \text{find}_{\mathbf{x}[\cdot], h} \quad \text{subject to} \quad & q[0] = 0, \quad \dot{q}[0] > 0, \\ & \mathbf{x}[N] = \mathbf{x}[0], \text{ (periodicity constraint)} \\ & \text{collocation dynamic constraints} \\ & 0.01 \leq h \leq 0.5 \end{aligned}$$

Try it yourself:



Launch in Deepnote

As always, make sure that you take a look at the code. Poke around. Try changing some things.

One of the things that you should notice in the code is that I provide an initial guess for the solver. In most of the examples so far I've been able to avoid doing that--the solver takes small random numbers as a default initial guess and solves from there. But for this problem, I found that it was getting stuck in a local minima. Adding the initial guess that the solution moves around a circle in state space was enough.

16.2 LYAPUNOV ANALYSIS

Recall the important distinction between stability of a trajectory in time and stability of a limit cycle was that the limit cycle does not converge in phase -- trajectories near the cycle converge to the cycle, but trajectories on the cycle may not converge with each other. This type of stability, also known as *orbital stability* can be written as stability to the manifold described by a trajectory $\mathbf{x}^*(t)$,

$$\min_{\tau} ||x(t) - x^*(\tau)|| \rightarrow 0.$$

In the case of limit cycles, this manifold is a periodic solution with $\mathbf{x}^*(t + t_{period}) = \mathbf{x}^*(t)$. Depending on exactly how that convergence happens, we can define orbital stability in the sense of Lyapunov, asymptotic orbital stability, exponential orbital stability, or even finite-time orbital stability.

In order to prove that a system is orbitally stable (locally, over a region, or globally), or to analyze the region of attraction of a limit cycle, we can use a Lyapunov function. When we analyzed our simple models of walking and running, we carried out the analysis on the Poincare map. Indeed, if we can find a Lyapunov function that proves (i.s.L., asymptotic, or exponential) stability of the discrete Poincare map, then this implies (i.s.L., asymptotic, or exponential) *orbital* stability of the cycle. But Lyapunov functions of this form are difficult to verify for a pretty fundamental reason: we rarely have an analytical expression for the Poincare map, since it is the result of integrating a nonlinear dynamics over the cycle.

Instead, we will focus our attention on constructing Lyapunov functions in the full state space, which use the continuous dynamics. In particular, we would like to consider Lyapunov functions which have the form cartooned below; they vanish to zero everywhere along the cycle, and are strictly positive everywhere away from the cycle.

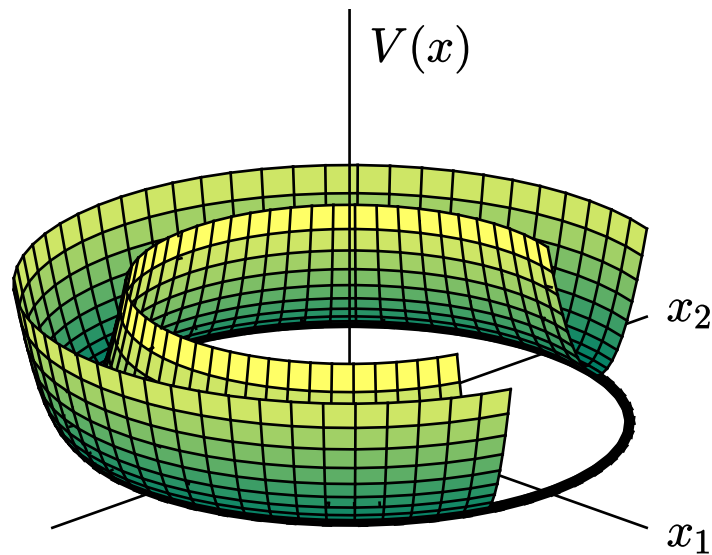


Figure 16.1 - Cartoon of a Lyapunov function which vanishes on a limit cycle, and is strictly positive everywhere else. (a small segment has been removed solely for the purposes of visualization).

16.2.1 Transverse coordinates

How can we parameterize this class of functions? For arbitrary cycles this could be very difficult in the original coordinates. For simple cycles like in the cartoon, one could imagine using polar coordinates. More generally, we will define a new coordinate system relative to the orbit, with coordinates

- τ - the phase along the orbit
- $\mathbf{x}_\perp(\tau)$ - the remaining coordinates, linearly independent from τ .

Given a state $\mathbf{x}$ in the original coordinates, we must define a smooth mapping $\mathbf{x} \rightarrow (\tau, \mathbf{x}_\perp)$ to this new coordinate system. For example, for a simple ring oscillator we might have:

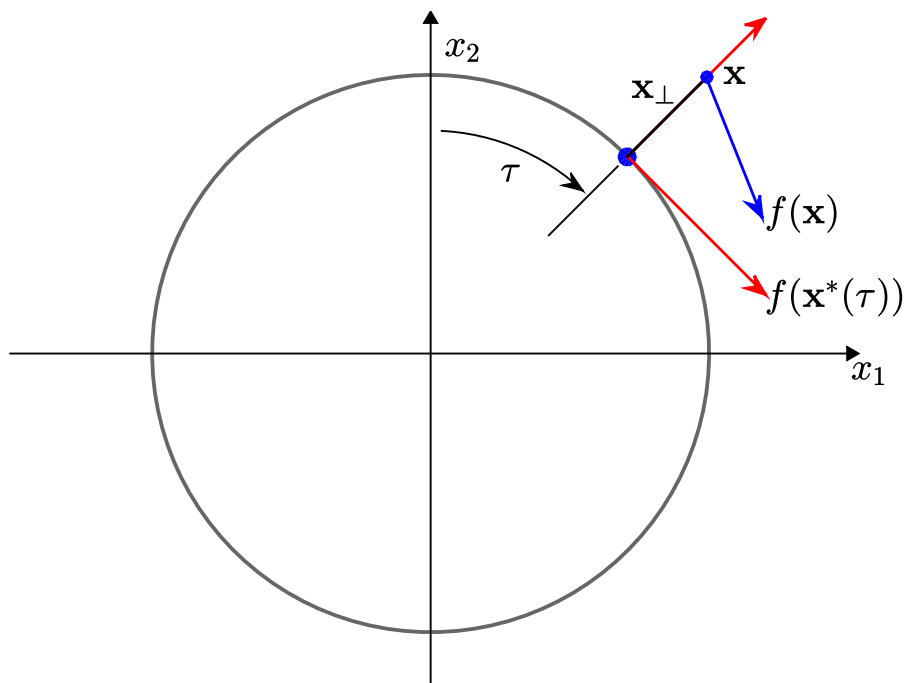


Figure 16.2 - A moving coordinate system along the limit cycle of the ring oscillator.

In general, for an n -dimensional state space, τ will always be a scalar, and $\mathbf{x}_\perp$ will be an $(n - 1)$ -dimensional vector defining the remaining coordinates relative to $\mathbf{x}^*(\tau)$. In fact, although we use the notation $\mathbf{x}_\perp$ the coordinate system need not be strictly orthogonal to the orbit, but must simply be *transversal* (not parallel). Having defined the smooth mapping $\mathbf{x} \rightarrow (\tau, \mathbf{x}_\perp)$, we can always rewrite the dynamics in this new coordinate system:

$$\begin{aligned}\dot{\tau} &= f_\tau(\mathbf{x}_\perp, \tau) \\ \dot{\mathbf{x}}_\perp &= f_\perp(\mathbf{x}_\perp, \tau).\end{aligned}$$

To keep our notation simpler for the remainder of these notes, we will assume that the origin of this new coordinate system is on the nominal trajectory ($\min_\tau |\mathbf{x} - \mathbf{x}^*(\tau)| = 0 \Rightarrow \mathbf{x}_\perp = 0$). Similarly, by taking τ to be the phase variable, we will leverage the fact that on the nominal trajectory, we have $\dot{\tau} = f_\tau(0, \tau) = 1$.

The value of this construction for Lyapunov analysis was proposed in [1] and has been extended nicely to control design in [2] and for region of attraction estimation in [3]. A quite general numerical strategy for defining the transverse coordinates is given in [4].

Theorem 16.1 - A Lyapunov theorem for orbital stability

For a dynamical system $\dot{\mathbf{x}} = f(\mathbf{x})$ with $\mathbf{x} \in \mathbb{R}^n$, f continuous, a continuous periodic solution $\mathbf{x}^*(\tau)$, and a smooth mapping $\mathbf{x} \rightarrow (\tau, \mathbf{x}_\perp)$ where $\mathbf{x}_\perp$ vanishes on $\mathbf{x}^*$, then for some $n - 1$ dimensional ball $\mathcal{B}$ around the origin, if I can produce a $V(\mathbf{x}_\perp, \tau)$ such that

$$\begin{aligned}\forall \tau, V(0, \tau) &= 0, \\ \forall \tau, \forall \mathbf{x}_\perp \in \mathcal{B}, \mathbf{x}_\perp \neq 0, V(\mathbf{x}_\perp, \tau) &> 0,\end{aligned}$$

with

$$\begin{aligned}\forall \tau, \dot{V}(0, \tau) &= 0, \\ \forall \tau, \forall \mathbf{x}_\perp \in \mathcal{B}, \mathbf{x}_\perp \neq 0, \dot{V}(\mathbf{x}_\perp, \tau) &< 0,\end{aligned}$$

then the solution $\mathbf{x}^*(t)$ is *locally orbitally asymptotically stable*.

Orbital stability in the sense of Lyapunov and exponential orbital stability can also be verified (with $\dot{V}_\perp \leq 0$ and $\dot{V}_\perp \leq \alpha V_\perp$, respectively).

Example 16.2 (Simple ring oscillator)

Perhaps the simplest oscillator is the first-order system which converges to the unit circle. In cartesian coordinates, the dynamics are

$$\begin{aligned}\dot{x}_1 &= x_2 - \alpha x_1 \left(1 - \frac{1}{\sqrt{x_1^2 + x_2^2}} \right) \\ \dot{x}_2 &= -x_1 - \alpha x_2 \left(1 - \frac{1}{\sqrt{x_1^2 + x_2^2}} \right),\end{aligned}$$

where α is a positive scalar gain.

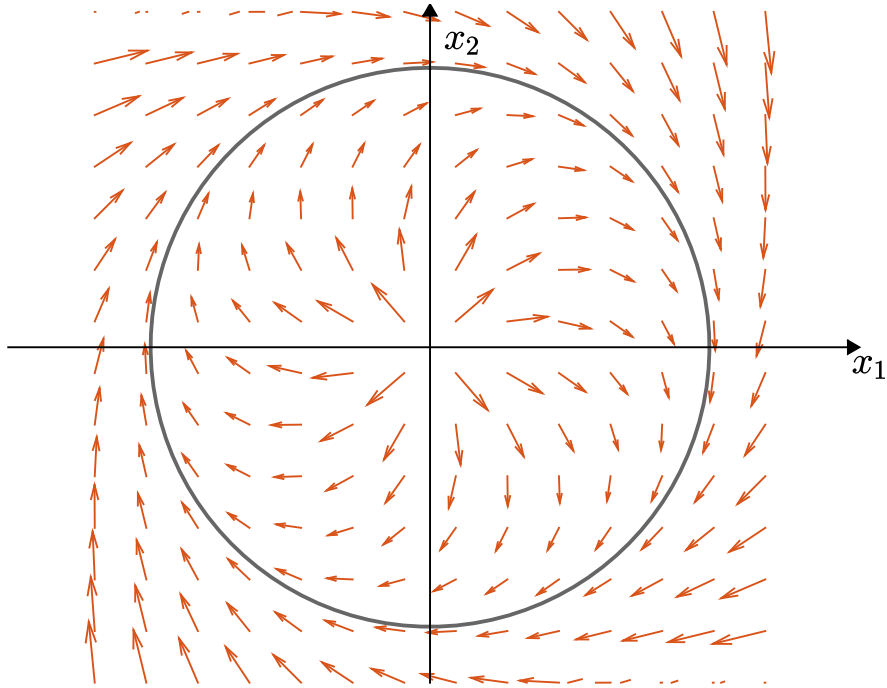


Figure 16.3 - Vector field of the ring oscillator

If we take the transverse coordinates to be the polar coordinates, shifted so that $x_{\perp}$ is zero on the unit circle,

$$\begin{aligned}\tau &= \text{atan2}(-x_2, x_1) \\ x_{\perp} &= \sqrt{x_1^2 + x_2^2} - 1,\end{aligned}$$

which is valid when $x_{\perp} > -1$, then the simple transverse dynamics are revealed:

$$\begin{aligned}\dot{\tau} &= 1 \\ \dot{x}_{\perp} &= -\alpha x_{\perp}.\end{aligned}$$

Taking a Lyapunov candidate $V(x_{\perp}, \tau) = x_{\perp}^2$, we can verify that

$$\dot{V} = -2\alpha x_{\perp}^2 < 0, \quad \forall x_{\perp} > -1.$$

This demonstrates that the limit cycle is locally asymptotically stable, and furthermore that the invariant open-set $V < 1$ is inside the region of attraction of that limit cycle. In fact, we know that all $x_{\perp} > -1$ are in the region of attraction that limit cycle, but this is not proven by the Lyapunov argument above.

Let's compare this approach again with the approach that we used in the chapter on walking robots, where we used a Poincaré map analysis to investigate limit cycle stability. In transverse coordinates approach, there is an additional burden to construct the coordinate system along the entire trajectory, instead of only at a single surface of section. In fact, the transverse coordinates approach is sometimes referred to as a "moving Poincaré section". Again, the reward for this extra work is that we can check a condition that only involves the instantaneous dynamics, $f(\mathbf{x})$, as opposed to having to integrate the dynamics over an entire cycle to generate the discrete Poincaré map, $\mathbf{x}_p[n+1] = P(\mathbf{x}_p[n])$. As we will see below, this approach will also be more compatible with designing continuous feedback controller that stabilize the limit cycle.

16.2.2 Transverse linearization

In the case of Lyapunov analysis around a fixed point, there was an important special case: for stable linear systems, we actually have a recipe for constructing a Lyapunov function. As a result, for nonlinear systems, we often found it convenient to begin our search by linearizing around the fixed point and using the Lyapunov candidate for the linear system as an initial guess. That same approach can be extended to limit cycle analysis.

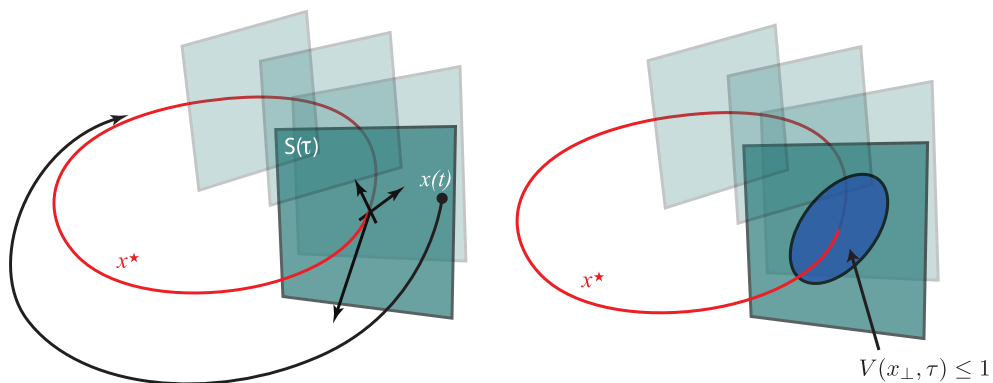


Figure 16.4 - Transverse surfaces and a Lyapunov function defined in the transverse coordinates (reproduced from [3]).

In particular, let us make a linear approximation of the transverse dynamics:

$$\dot{\mathbf{x}}_{\perp} = f_{\perp}(\mathbf{x}_{\perp}, \tau) \approx \frac{\partial f_{\perp}}{\partial \mathbf{x}_{\perp}} \mathbf{x}_{\perp} = \mathbf{A}_{\perp}(\tau) \mathbf{x}_{\perp}.$$

Note that I've assumed that $\mathbf{x}_{\perp}$ is zero on the nominal trajectory, so don't need the additional notation of working in the error coordinates here. Remember that τ is the

phase along the trajectory, but [1] showed that (exponential) stability of the *time-varying* linear system $\dot{\mathbf{x}}_{\perp} = \mathbf{A}_{\perp}(t)\mathbf{x}_{\perp}$ implies local exponential orbital stability of the original system. In particular, if this transverse linear system is periodic and orbitally stable, then for each $\mathbf{Q} = \mathbf{Q}^T \succ 0$, there exists a unique positive definite solution to the periodic Riccati equation:

$$V(\mathbf{x}_{\perp}, \tau) = \mathbf{x}_{\perp}^T \mathbf{P}(\tau) \mathbf{x}_{\perp}, \quad \mathbf{P}^T(\tau) = \mathbf{P}(\tau), \quad \mathbf{P}(\tau + t_{\text{period}}) = \mathbf{P}(\tau), \\ -\dot{\mathbf{P}}(\tau) = \mathbf{P}(\tau)\mathbf{A}(\tau) + \mathbf{A}(\tau)\mathbf{P}^T(\tau) + \mathbf{Q}, \quad \mathbf{Q} = \mathbf{Q}^T \succ 0.$$

There is a surprisingly rich literature on the numerical methods for finding the periodic solutions to these Lyapunov (and corresponding Riccati) equations. In practice, for every problem I've ever tried, I've simply integrated the equations backwards in time until the integration converges to a periodic solution (this is not guaranteed to work, but almost always does).

This is very powerful. It gives us a general way to construct a Lyapunov candidate that certifies local stability of the cycle, and which can be used as a candidate for nonlinear Lyapunov analysis.

16.2.3 Region of attraction estimation using sums-of-squares

Coming soon. If you are interested, see [4, 3].

16.3 FEEDBACK DESIGN

Many of the tools we've developed for stabilization of fixed points or stabilization of trajectories can be adapted to this new setting. When we discussed control for the spring-loaded inverted pendulum model, we considered discrete control decisions, made once per cycle, which could stabilize the discrete Poincare map, $\mathbf{x}_p[n+1] = f_p(\mathbf{x}_p[n], \mathbf{u}[n])$, with $\mathbf{u}[n]$ the once-per-cycle decisions. While it's possible to get the local linear approximations of this map using our adjoint methods, remember that we rarely have an analytical expression for the nonlinear f_p .

How limiting is it to restrict ourselves to once-per-cycle decisions? Of course we can improve performance in general with continuous feedback. But there are also some cases where once-per-cycle decisions feel quite reasonable. Foot placement for legged robots is a good example -- once we've chosen a foot placement we might make minor corrections just before we place the foot, but rarely change that decision once the foot is on the ground; once per footstep seems like a reasonable approximation. Another good example is flapping flight: here the wing beat cycle times can be much faster than the dynamic timescales of the center of mass, and changing the parameters of the wing stroke one per flap seems pretty reasonable.

But the tools we've developed above also give us the machinery we need to consider continuous feedback throughout the trajectory. Let's look at a few important formulations.

16.3.1 For underactuation degree one.

It turns out many of our simple walking models -- particularly ones with a point foot that are derived in the minimal coordinates -- are only short one actuator (between the foot and the ground). One can represent even fairly complex robots this way; much of the theory that I'll elude to here was originally developed by Jessy Grizzle and his group in the context of the bipedal robot [RABBIT](#). Jessy's key observation was that limit cycle stability is effectively stability in $n - 1$ degrees of freedom, and you can often achieve it easily with $n - 1$ actuators -- he called this line of work "Hybrid Zero Dynamics" (HZD). We'll deal with the "hybrid" part of that in the next chapter, but here is a simple example to illustrate the "zero dynamics" concept. [Coming soon...]

Stabilizing the zero dynamics is synonymous with achieving orbital stability. Interestingly, it doesn't actually guarantee that you will go around the limit cycle. One "feature" of the HZD walkers is that you can actually stand in front of them and stop their forward progress with your hand -- the control system will continue to act, maintaining itself on the cycle, but progress along the cycle has stopped. In some of the robots you could even push them backwards and they would move through the same walking cycle in reverse. If one wants to certify that forward progress will be achieved, then the task is reduced to inspecting the one-dimensional dynamics of the phase variable along the cycle.

The notion of "zero dynamics" is certainly not restricted to systems with underactuation of degree one. In general, we can easily stabilize a manifold of dimension m with m actuators (we saw this in the section on [task-space partial feedback linearization](#)), and if being on that manifold is sufficient to achieve our task, or if we can certify that the resulting dynamics on the manifold are sufficient for our task, then life is good. But in the "underactuation degree one" case, the manifold under study is a trajectory/orbit, and the tools from this chapter are immediately applicable.

16.3.2 Transverse LQR

Another natural approach to stabilizing a cycle uses the transverse linearization. In particular, for a nominal (control) trajectory, $[\mathbf{x}_0(t), \mathbf{u}_0(t)]$, with $\dot{\mathbf{x}}_0 = f(\mathbf{x}_0, \mathbf{u}_0)$, we can approximate the transverse dynamics:

$$\dot{\mathbf{x}}_{\perp} = f_{\perp}(\mathbf{x}_{\perp}, \tau, \mathbf{u}) \approx \frac{\partial f_{\perp}}{\partial \mathbf{x}_{\perp}} \mathbf{x}_{\perp} + \frac{\partial f_{\perp}}{\partial \mathbf{u}} (\mathbf{u} - \mathbf{u}_0(\tau)) = \mathbf{A}_{\perp}(\tau) \mathbf{x}_{\perp} + \mathbf{B}_{\perp}(\tau) \bar{\mathbf{u}}.$$

[2] showed that the (periodic) time-varying LQR controller that stabilized this system achieves *orbital* stabilization of the original system.

Let's take a minute to appreciate the difference between this approach and time-varying LQR in the full coordinates. Of course, the behavior is quite different: time-varying LQR in the full coordinates will try to slow down or speed up to keep time with

the nominal trajectory, where this controller makes no attempt to stabilize the phase variable. You might think you could design the original time-varying controller, $\bar{\mathbf{u}} = \mathbf{K}(t)\bar{\mathbf{x}}$, and just "project to the closest time" during execution (e.g. find the time $t = \operatorname{argmin}_{\tau} \|\mathbf{x} - \mathbf{x}_0(\tau)\|$) and use the feedback corresponding to that time. But this projection is not guaranteed to be safe; you can find systems that are stabilizable by the transverse linearization are unstable in closed-loop using a feedback based on this projection.

Example 16.3 (Time-varying LQR simply projected in time can lead to instability.)

Coming soon...

But there is an even more important reason to like the transverse LQR approach. If orbital stability is all you need, then this is a better formulation of the problem because you are asking for less. As we've just discussed with hybrid zero-dynamics, stabilizing m degrees of freedom with m actuators is potentially very different than stabilizing $m + 1$ degrees of freedom. In the transverse formulation, we are asking LQR to minimize the cost only in the transverse coordinates (it is mathematically equivalent to designing a cost function in the full coordinates that is zero in the direction of the nominal trajectory). In practice, this can result in much smaller cost-to-go matrices, $\mathbf{S}(t)$, and smaller feedback gains, $\mathbf{K}(t)$. For underactuated systems, this difference can be dramatic.

16.3.3 Orbital stabilization for non-periodic trajectories

The observation that orbital stabilization of a trajectory can ask much less of your underactuated control system (and lead to smaller LQR feedback gains), is quite powerful. It is not limited to stabilizing periodic motions. If your aim is to stabilize a non-periodic path through state-space, but do not actually care about the timing, then formulating your stabilization can be a very good choice. You can find examples where a system is stabilizable in the transverse coordinates, but not in the full coordinates.

Example 16.4 (Stabilizable in the transverse coordinates, but not the full coordinates.)

Coming soon...

Take care, though, as the converse can also be true: it's possible that a system could be stabilizable in the full coordinates but *not* in some transverse coordinates; if you choose those coordinates badly.

Example 16.5 (Stabilizable in the full coordinates, but not the transverse coordinates.)

Consider a trajectory with $u(t) = 1$ (from any initial condition) for [the double integrator](#). We can certainly stabilize this trajectory in the full coordinates using LQR. Technically, one can choose transverse coordinates $\tau = \dot{q}$, and $x_{\perp} = q$. Clearly $x_{\perp}$ is transverse to the nominal trajectory everywhere. But this transverse linearization is also clearly not stabilizable. It's a bad choice!

The general approach to designing the transverse coordinates [4] could address this concern by including a criteria for controllability when optimizing the transverse coordinates.

REFERENCES

1. John Hauser and Chung Choo Chung, "Converse Lyapunov functions for exponentially stable periodic orbits", *Systems & Control Letters*, vol. 23, no. 1, pp. 27 -- 34, 1994.
2. Anton S. Shiriaev and Leonid B. Freidovich and Ian R. Manchester, "Can We Make a Robot Ballerina Perform a Pirouette? Orbital Stabilization of Periodic Motions of Underactuated Mechanical Systems", *Annual Reviews in Control*, vol. 32, no. 2, pp. 200--211, Dec, 2008.
3. Ian R. Manchester and Mark M. Tobenkin and Michael Levashov and Russ Tedrake, "Regions of Attraction for Hybrid Limit Cycles of Walking Robots", *Proceedings of the 18th IFAC World Congress, extended version available online: arXiv:1010.2247 [math.OC]*, 2011.
4. Ian R. Manchester, "Transverse Dynamics and Regions of Stability for Nonlinear Hybrid Limit Cycles", *Proceedings of the 18th IFAC World Congress, extended version available online: arXiv:1010.2241 [math.OC]*, Aug-Sep, 2011.

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

[Accessibility](#)

© Russ Tedrake, 2024